Churn Risk Prediction MVP

Architecture and Data Pipeline Report

Gadiel Mora

October 2026

1 Executive Summary

This document describes the architecture and data pipeline of a churn-prediction MVP built as a proof-of-work demonstration for an AI/ML consulting pitch to a cybersecurity company in Ecuador. The system predicts which clients of a subscription-based service are at risk of cancelling, so a company can intervene before losing them.

Since no real client data exists yet, the pipeline currently runs on the **IBM Telco Customer Churn** dataset (via Kaggle) as a structural proxy: it mirrors the shape a subscription-based security service's client data would take (contract length, monthly cost, support interactions, cancellation flag). The architecture is designed so that swapping in a real client's data later requires no structural changes — only pointing the ingestion stage at a new source.

2 System Architecture

The system follows a *medallion* data architecture (bronze → silver → gold) feeding a trained model, served through an authenticated API, and visualized in a web dashboard.

3 The Data Pipeline: How the IBM Dataset Flows Through the System

3.1 What the source data is

The **IBM Telco Customer Churn** dataset (Kaggle: `yeanzc/telco-customer-churn-ibm-dataset`) describes 7,043 fictional customers of a California telecom provider, with 33 columns covering demographics, subscribed services, billing, and churn outcome. It is used here purely as a *structural analog* for a subscription-based security service — not as real client data.

3.2 Step 1 — Bronze: raw ingestion

`src/pipeline/ingest.py` authenticates against the Kaggle API and downloads the dataset unmodified into `data/bronze/`. The file arrives as an Excel spreadsheet (`.xlsx`), not CSV — a detail only discovered by actually running the download, since Kaggle's listed format is not guaranteed. No transformation happens at this stage; bronze is a faithful copy of the source.

3.3 Step 2 — Silver: cleaning

`src/pipeline/transform.py` performs the minimum work needed to make the data analyzable without discarding any signal:

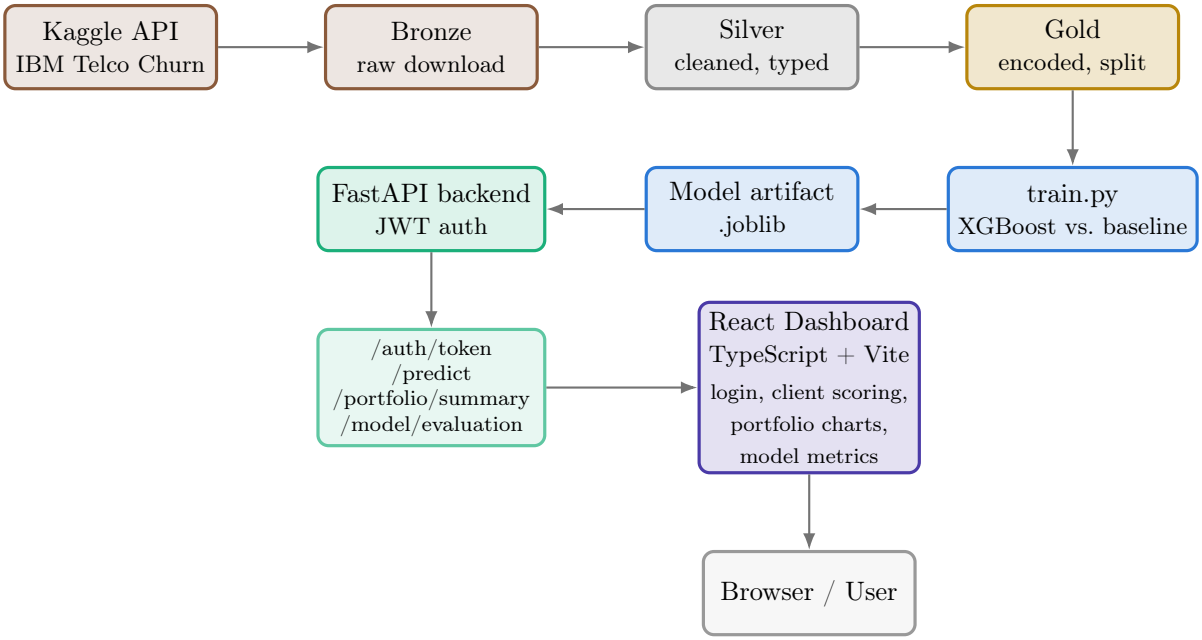


Figure 1: End-to-end architecture: raw data is pulled, cleaned, and encoded through three pipeline stages; the gold-stage data trains an XGBoost model; the trained model is wrapped by an authenticated FastAPI service; a React dashboard consumes that API.

- Column names are normalized to `snake_case` (e.g. `"Total Charges"` → `total_charges`).
- `total_charges` arrives as text (blank for brand-new customers with $0 billed) and is coerced to numeric; rows that fail coercion are dropped.
- Duplicate customer records (by `customer_id`) are removed.

The result (7,032 rows after cleaning) is written to `data/silver/customers_clean.parquet`, still containing *every* original column.

3.4 Step 3 — Gold: feature curation and leakage removal

`src/pipeline/gold.py` is where modeling decisions are made. This is the most consequential stage, because the raw IBM dataset contains several columns that would silently invalidate a model if left in:

Column	Why it is dropped	Category
`Churn Score`	IBM's own pre-computed churn-prediction output (0–100, via SPSS Modeler)	Leakage
`Churn Reason`	Free text, populated *only* for customers who already churned — directly reveals the label	Leakage
`CLTV`	Proprietary computed index, not a raw operational feature	Leakage
`Churn Label`	Duplicates `Churn Value` (the chosen target)	Redundant
`Country`, `State`, `Count`	Constant across all 7,032 rows	No signal
`City`, `Zip Code`, `Lat Long`, etc.	High-cardinality geographic identifiers (1,129 / 1,652 unique values) unsuitable for one-hot encoding at this scale	Noise

After removing these, the remaining 19 raw features (contract type, tenure, monthly/total charges, service add-ons, payment method, demographics) are one-hot encoded, expanding to **30 model-ready features**.

Finally, the data is split **70/15/15 into train / validation / test**, stratified so each split preserves the dataset's ≈26.6% churn rate:

Split	Rows	Churn rate
Train	4,922 (70%)	26.57%
Validation	1,055 (15%)	26.64%
Test	1,055 (15%)	26.54%

Why three splits: the model learns from **train**; hyperparameters are tuned by repeatedly checking **validation**; **test** is touched exactly once, after every decision is locked in, to produce an honest final score uncontaminated by the tuning process.

4 Model Training

`src/models/train.py` fits a Logistic Regression baseline and an XGBoost classifier (small grid over `max_depth`, `n_estimators`, `learning_rate`, selected by validation ROC-AUC). XGBoost was chosen over a recurrent neural network because churn here is a **tabular** problem — one row per client — not a sequence problem; RNNs earn their complexity on ordered sequential data (e.g. raw event logs per client over time), which this is not.

Final, single-use evaluation on the held-out test set:

Metric	Value
ROC-AUC	0.835
Precision (churn class)	51%
Recall (churn class)	81%
F1 score	0.622

High recall was prioritized over precision: failing to flag an actual churner (a silent lost client) is more costly than a false alarm (checking in on a healthy client).

5 Serving and Dashboard

The trained model is wrapped by a **FastAPI** backend (JWT-authenticated) exposing:

- `POST /auth/token` — login, issues a 30-minute bearer token
- `POST /predict` — scores a single client profile
- `GET /portfolio/summary` — aggregate risk distribution and top churn drivers across the client base
- `GET /model/evaluation` — ROC curve, confusion matrix, and headline metrics

A **React + TypeScript** dashboard consumes this API: a login screen, a portfolio overview (risk distribution and feature-importance charts), model-performance visualizations (ROC curve, confusion matrix), and a form to score an individual client.

6 Current Status and Next Steps

The full pipeline, model, API, and dashboard are built and verified working locally. Two items remain explicitly deferred:

- **Real client data** — swapping the Kaggle proxy for an actual client's CRM/billing/ticketing export, gated on securing that first client and a data-sharing agreement. The pipeline's structure does not change; only the source and the leakage/noise column audit need to be redone against the real schema.
- **Lead generation** — kept as a manual research process (not an in-app feature), since no free bulk-accessible dataset of Ecuadorian companies by sector was found to exist.